

BEACONHOUSE NATIONAL UNIVERSITY



Assignment 01

Name: Saad Mughal

BNU ID: F2023-009

Total Marks: 10

Due Date: 19-10-2025

AoA(SEC_C)

Course Instructor	Sir Zaman Aziz
Semester	Fall 2025

19-Oct-25

Assignment # 1 ANALYSIS OF ALGORITHMS TIME COMPLEXITY

SUNDAY

Q1. Calculate time complexity (Best and Worst case both.)

```

void B.S (int arr[], int n) {
1.   for (int i = 0; i < n - 1; i++) {
2.       for (int j = 0; j < n - i - 1; j++) {
3.           if (arr[j] > arr[j+1]) {
4.               int temp = arr[j];
5.               arr[j] = arr[j+1];
6.               arr[j+1] = temp;
           }
       }
   }
}

```

$\frac{n(n-1)}{2}$ Bubble Sort

Bubble Sort Logic :

e.g; iteration = 1

let arr [6 3 9 1] n = 4

↓ ↓
1 [6 | 3 | 9 | 1]

↓ ↓
2 [3 | 6 | 9 | 1]

↓ ↓
3 [3 | 6 | 9 | 1]

↓
[3 | 6 | 1 | 9]

n = 4, n - 1 = 3 (comparisons)

Iterations	Comparison
1 st	(n-1)
2 nd	(n-2)
3 rd	(n-3)
⋮	⋮
Last	1

For Array → n elements

$$= (n-1) + (n-2) + (n-3) + \dots + 1$$

$$= \frac{n(n-1)}{2} \approx n^2$$

$$\text{Loops} = 2 = n * n = n^2$$

Q1:

0	1	2	3
6	3	9	1

when $j = n-1$

→ pass 1 ($i \leq 0$)

$j = 0$

compare $arr[0] > arr[1]$

($6 > 3$)

SWAP [3 6 9 1]

$j = 1$

compare $arr[1] > arr[2]$

($6 > 9$)

False (no SWAP)

$j = 2$

compare $arr[2] > arr[3]$

($9 > 1$)

SWAP [3 6 1 9]

← →

→ pass 2 ($i \leq 1$)

$j = 0$

compare $arr[0] > arr[1]$

False (no SWAP) ($3 > 6$)

$j = 1$

compare $arr[1] > arr[2]$

SWAP [3 1 6 9] ($6 > 1$)

← →

→ pass 3 ($i \leq 2$)

$j = 0$

compare $arr[0] > arr[1]$

SWAP [1 3 6 9]

↑ ($3 > 1$)

ops	1	0
C_1	n	n
C_2	$\frac{n(n-1)}{2}$	$\frac{n(n-1)}{2}$
C_3	$\frac{n(n-1)}{2}$	$\frac{n(n-1)}{2}$
C_4	0	$\frac{n(n-1)}{2}$
C_5	0	$\frac{n(n-1)}{2}$
C_6	0	$\frac{n(n-1)}{2}$

Time Complexity:

Best: $\Omega(n^2)$

Worst: $O(n^2)$

Q2: Solve problem in $O(\log n)$

Pseudo code

function findFirst(arr, key, low, high)

result = -1

while low $\leq$ high

mid = low + (high - low) / 2

if arr[mid] == key:

result = mid

high = mid - 1

else if arr[mid] < key:

low = mid + 1

else:

high = mid - 1

return result

function findSecondOccurrence(arr, n, key):

first = findFirst(arr, key, 0, n-1)

if first == -1

return "key not found"

else:

second = findFirst(arr, key, first+1, n-1)

if second == -1

return "2nd occurrence doesn't exist"

else:

return "2nd occurrence at index", second

Time Complexity:

Two binary searches $\rightarrow O(\log n) + O(\log n) = O(\log n)$

Let arr = [2 4 4 4 6 7 8] where key = 4

1) Find first occurrence of 4

$\rightarrow$ low = 0, high = 6

mid = 3 $\rightarrow$ arr[3] = 4 $\rightarrow$ found, set result = 3, high = 2

$\rightarrow$ low = 0, high = 2

mid = 1 $\rightarrow$ arr[1] = 4 $\rightarrow$ found, result = 1, high = 0

$\rightarrow$ low = 0, high = 0

mid = 0 $\rightarrow$ arr[0] = 2 < 4 $\rightarrow$ low = 1

$\rightarrow$ low > high $\rightarrow$ stop

first occurrence = [index] = 1

2) Find first occurrence from Index 2

$\rightarrow$ low = first + 1 = 2, high = 6

mid = 4 $\rightarrow$ arr[4] = 6 > 4 $\rightarrow$ high = 3

$\rightarrow$ low = 2, high = 3

mid = 2 $\rightarrow$ arr[2] = 4 $\rightarrow$ found, result = 2, high = 1

$\rightarrow$ low > high $\rightarrow$ stop

second occurrence Index = 2

Output = Second occurrence at Index = 2

key appears once - example:

Let arr [1 3 5 7] where key = 3

First Occurrence = 1

Search from Index 2 to 3 for 3 $\Rightarrow$ NOT FOUND

Output: Second occurrence doesn't exist

key not present - example:

Let arr [1 3 5 7] where key = 4

First occurrence = -1

Output = key not found